

Sri Lanka Institute of Information Technology



Module Code: IT2011

Module Name: Artificial Intelligence and Machine Learning

Calendar Year: Year 2

Semester: 1

Group Number:156

Group ID: 2025-Y2-S1-MLB-B8G2-06

Project Topic Name: Students Performance Dataset

Group Members	ID Number
Dhanodya G.G. R	IT24102297
Bambarende S C	IT24102225
Anushan L.K.D.	IT24102306
Mohammed M H S	IT24102308
Punchihewa D S R	IT24102298
Senadheera P.V.S.T.	IT24102257

1. INTRODUCTION AND PROBLEM STATEMENT

Introduction

In today’s educational environment, understanding what influences student success is crucial for improving academic performance. With advancements in AI and ML, it is now possible to analyze patterns in student data and make accurate predictions about their academic outcomes.

This project aims to develop a machine learning model that predicts a student’s overall academic performance based on various factors. By analyzing these variables, educators can identify students who may be at risk of underperforming and help them to enhance learning.

Problem Statement

Educational institutions often struggle to recognize students who are likely to perform poorly until it is too late to offer effective help. Traditional evaluation methods focus mainly on exam scores and attendance, ignoring other influential factors like stress, sleep, and family background.

The goal of this system is to analyze multiple variables and predict the **Total Score** of each student.

2. DATABASE DESCRIPTION

Dataset Overview

- **Dataset Name:** Students Performance Dataset
- **File Type:** CSV (Comma-Separated Values)
- **Data Type:** Tabular (Structured Data)
- **Purpose:** Train a machine learning model that predicts the total score of students.

Dataset Features

The dataset contains **22 features**, covering academic, behavioral & lifestyle factors of students.

Feature Name	Type
Student_ID	Categorical
First_Name	Text
Last_Name	Text
Email	Text
Gender	Categorical
Age	Numerical
Department	Categorical
Attendance (%)	Numerical

Feature Name	Type
Participation_Score	Numerical
Projects_Score	Numerical
Total_Score	Numerical
Grade	Categorical
Study_Hours_per_Week	Numerical
Extracurricular_Activities	Categorical
Internet_Access_at_Home	Categorical
Midterm_Score	Numerical

Feature Name	Type
Parent_Education_Level	Categorical
Final_Score	Numerical
Assignments_Avg	Numerical
Family_Income_Level	Categorical
Quizzes_Avg	Numerical
Stress_Level (1–10)	Numerical
Sleep_Hours_per_Night	Numerical

3. PREPROCESSING AND EXPLORATORY DATA ANALYSIS (EDA)

Data Cleaning and Missing Value Handling

Filling Missing Values:

- For numerical columns, missing values were filled using the mean of each column.
- For categorical columns, missing values were filled using the most frequent value (also known as the mode) in each column.

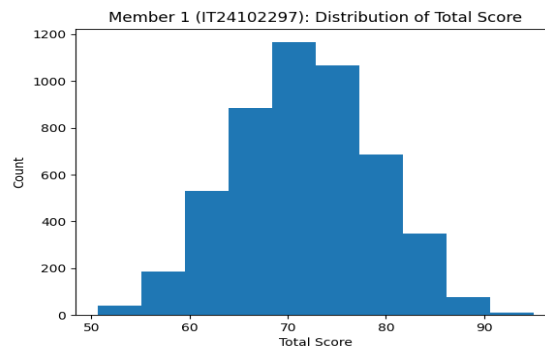
```
num_cols = df.select_dtypes(include=['float64', 'int64']).columns
cat_cols = df.select_dtypes(include=['object']).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].mean())
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])
```

Missing values before imputation:

Student_ID	0
First_Name	0
Last_Name	0
Email	0
Gender	0
Age	0
Department	0
Attendance (%)	0
Midterm_Score	0
Final_Score	0
Assignments_Avg	0
Quizzes_Avg	0
Participation_Score	0
Projects_Score	0
Total_Score	0
Grade	0
Study_Hours_per_Week	0
Extracurricular_Activities	0
Internet_Access_at_Home	0
Parent_Education_Level	1025
Family_Income_Level	0
Stress_Level (1-10)	0
Sleep_Hours_per_Night	0

dtype: int64

Missing values after imputation:
0 total missing values remaining (should be 0)
Saved: imputed_data.csv



Feature Encoding

1. One-Hot Encoding

One-hot encoding was applied to the 'Gender' and 'Department' columns. This creates a new binary column for each unique category in those columns.

- For example, if the 'Gender' column has values "Male" and "Female", two new columns will be created: Gender_Male and Gender_Female.
- Similarly, separate columns were created for each department in the 'Department' column.

```
# One-Hot Encoding
df = pd.get_dummies(df, columns=['Gender', 'Department'], drop_first=False)
```

2. Label Encoding

Label encoding was applied to ordinal categorical columns, where the order of values matters. Specific mappings were created manually to reflect this order:

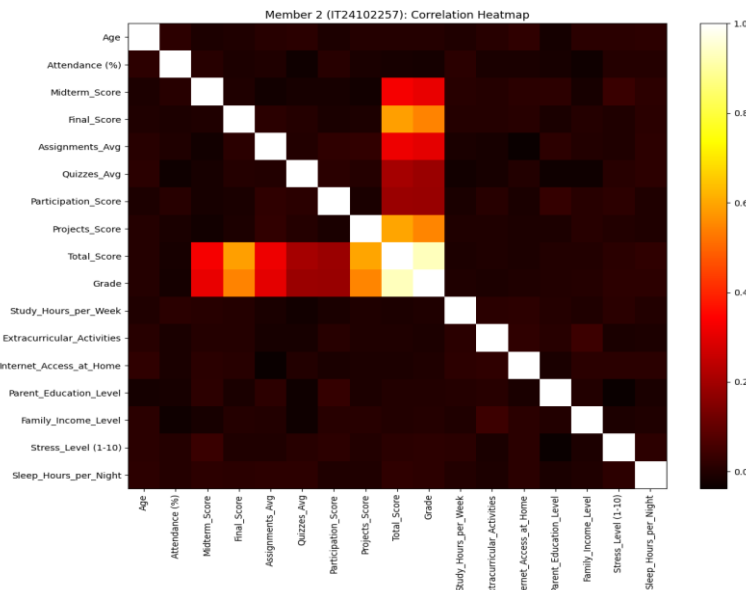
- **Grade:**
 - 'A' → 4
 - 'B' → 3
 - 'C' → 2
 - 'D' → 1
 - 'F' → 0
- **Parent's Education Level:**
 - 'High School' → 0
 - "Bachelor's" → 1
 - "Master's" → 2
 - 'PhD' → 3
- **Family Income Level:**
 - 'Low' → 0
 - 'Medium' → 1
 - 'High' → 2

- **Binary Yes/No Columns (like 'Extracurricular Activities' and 'Internet Access at Home'):**

- 'Yes' → 1
- 'No' → 0

```
# Label Encoding
grade_order = {'A':4, 'B':3, 'C':2, 'D':1, 'F':0}
df['Grade'] = df['Grade'].map(grade_order)
education_order = {'High School':0, "Bachelor's":1, "Master's":2, 'PhD':3}
df['Parent_Education_Level'] = df['Parent_Education_Level'].map(education_order)
income_order = {'Low':0, 'Medium':1, 'High':2}
df['Family_Income_Level'] = df['Family_Income_Level'].map(income_order)
df['Extracurricular_Activities'] = df['Extracurricular_Activities'].map({'No':0, 'Yes':1})
df['Internet_Access_at_Home'] = df['Internet_Access_at_Home'].map({'No':0, 'Yes':1})
```

	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB	A
1	Extracurric	Internet_A	Parent_Edu	Family_Inc	Stress_Lev	Sleep_Hou	Gender_Fe	Gender_M	Departme	Departme	Departme	Department	Mathematics	
2	1	0	2	1	1	5.9	TRUE	FALSE	FALSE	FALSE	FALSE	TRUE		
3	0	0	0	0	4	4.3	FALSE	TRUE	TRUE	FALSE	FALSE	FALSE		
4	1	0	0	0	9	6.1	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE		
5	0	1	0	0	8	4.9	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE		
6	1	0	2	1	6	4.5	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE		
7	1	0	1	1	6	5.6	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE		
8	1	0	1	2	2	4	FALSE	TRUE	TRUE	FALSE	FALSE	FALSE		



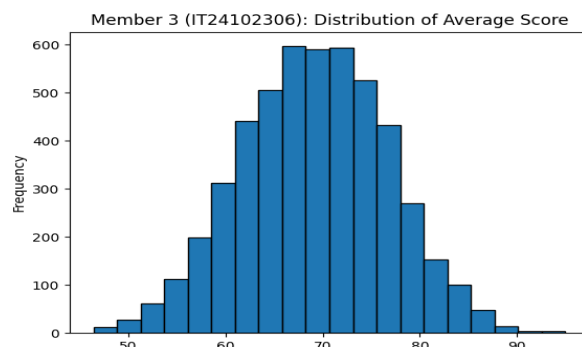
Feature Engineering: Creating New Features

To improve model performance and provide deeper insights, new features were created from existing data. These engineered features help capture more meaningful patterns and relationships in the dataset.

1. Average Score

This feature represents the average academic performance of a student, calculated by taking the mean of several key academic metrics:

```
# Create new features
# 1. Average_Score: Mean of academic performance metrics
df['Average_Score'] = df[['Midterm_Score', 'Final_Score', 'Assignments_Avg', 'Quizzes_Avg', 'Participation_Score', 'Projects_Score']].mean(axis=1)
```



2. Study_Efficiency

This feature measures how effectively a student converts their study time into academic results. It is calculated as: $\text{Study_Efficiency} = \text{Total Score} / \text{Study Hours per Week}$

```
# 2. Study_Efficiency: Total_Score / Study_Hours_per_Week (handle zero division)
df['Study_Efficiency'] = df.apply(lambda x: x['Total_Score'] / x['Study_Hours_per_Week'] if x['Study_Hours_per_Week'] != 0 else 0, axis=1)
```

3. Stress_Sleep_Ratio

This feature indicates the relationship between a student's stress level and the amount of sleep they get per night. It is calculated as:

$$\text{Stress_Sleep_Ratio} = \text{Stress Level} / \text{Sleep Hours per Night}$$

```
# 3. Stress_Sleep_Ratio: Stress_Level / Sleep_Hours_per_Night (handle zero division)
df['Stress_Sleep_Ratio'] = df.apply(lambda x: x['Stress_Level (1-10)'] / x['Sleep_Hours_per_Night'] if x['Sleep_Hours_per_Night'] != 0 else 0, axis=1)
```

1	Average_Si	Study_Effic	Stress_Sleep_Ratio
2	60.55333	5.814223	0.169492
3	81.66	3.022768	0.930233
4	56.24333	5.461048	1.47541
5	47.35667	2.025627	1.632653
6	69.71167	5.368647	1.333333
7	66.20833	2.691304	1.071429
8	71.795	3.749208	0.5

Outlier Detection and Treatment

To improve the quality of the data and reduce the effect of extreme values, outliers were identified and removed using the Interquartile Range (IQR) method for each numerical column.

Calculate Q1 and Q3:

- Q1 = 25th percentile (lower quartile)
- Q3 = 75th percentile (upper quartile)

Compute IQR (Interquartile Range):

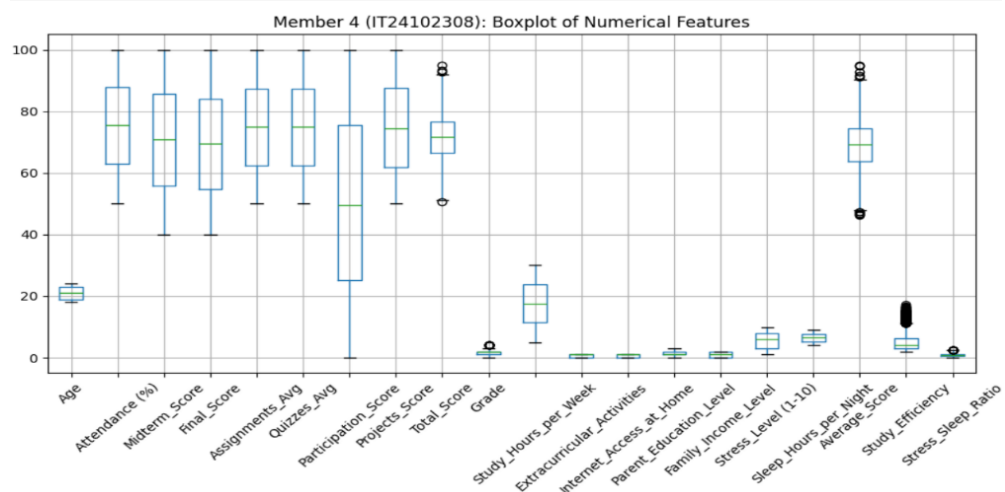
- $\text{IQR} = \text{Q3} - \text{Q1}$

Define Lower and Upper Limits:

- Lower limit = $\text{Q1} - 1.5 \times \text{IQR}$
- Upper limit = $\text{Q3} + 1.5 \times \text{IQR}$

Remove Outliers

Any data points below the lower limit or above the upper limit were considered outliers and removed from the dataset.



Scaling

To ensure that all numerical features contribute equally to the model and are on a comparable scale, standardization was applied using Z-score scaling.

$$Z = \frac{X - \mu}{\sigma}$$

Where:

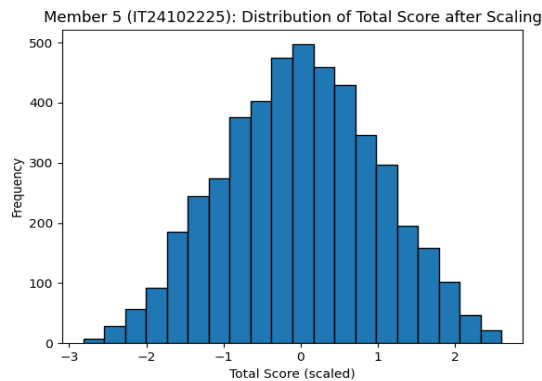
- X = original value
- μ = mean of the column
- σ = standard deviation of the column

```
# Apply Standardization (Z-score scaling)
```

```
scaler = StandardScaler()
```

```
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
```

Age	Attendance	Midterm_Score	Final_Score	Assignments_Avg	Quizzes_Avg	Participation_Score	Projects_Score	Total_Score	Grade	Study_Hours	Extracurricular	Internet_Access	Parent_Education	Family_Income	Stress_Level	Sleep_Hours
0.483316	1.523671	-1.71409	-0.56479	-0.07386	-1.49252	0.818181	-0.79847	-1.61554	-2.14487	-1.15634	0.993632	-1.00684	0.56966	0.010451	-1.56144	-0.42724
-1.52567	1.547971	-0.76013	0.277392	-0.03625	1.63029	1.321913	1.632982	1.474944	1.769244	1.306756	-1.00641	-1.00684	-1.40169	-1.21645	-0.52009	-1.53364
1.487809	1.673634	-1.64366	-0.31664	0.773067	-1.71222	-1.55212	1.151363	-0.51706	-0.84017	-0.84845	0.993632	-1.00684	-1.40169	-1.21645	1.215503	-0.28894
0.985563	-1.1111	-0.99719	-0.43838	-0.49384	0.642716	0.504211	0.890972	1.87E-05	0.464538	-0.7165	0.993632	-1.00684	0.56966	0.010451	0.17415	-1.39534
-0.01893	-1.02848	-0.01918	-0.27977	0.912363	-1.55559	0.17644	-0.468	-0.46474	-0.84017	1.042853	0.993632	-1.00684	-0.41602	0.010451	0.17415	-0.63469
1.487809	-0.58345	1.162668	1.43269	0.992459	-1.42322	-1.1795	0.896469	1.606879	1.769244	0.573692	0.993632	-1.00684	-0.41602	1.237354	-1.21432	-1.74109
-1.02342	-0.65982	1.212485	-0.57299	-1.67368	-1.44678	0.17644	1.070291	-0.00482	0.464538	-0.21802	0.993632	0.993209	-0.41602	-1.21645	-0.17297	0.88661



Dimensionality Reduction

To improve model performance and reduce complexity, the most relevant features were selected using the SelectKBest method.

```
# Feature Selection: SelectKBest
X = df.drop(['Total_Score', 'Grade'], axis=1)
y = df['Total_Score']
selector = SelectKBest(score_func=f_regression, k=8)
X_selected = selector.fit_transform(X, y)
selected_features = X.columns[selector.get_support()].tolist()
```

k=8 means the top 8 features will be selected.

```
Selected features: ['Midterm_Score', 'Final_Score', 'Assignments_Avg', 'Quizzes_Avg', 'Participation_Score', 'Projects_Score', 'Average_Score', 'Study_Efficiency']
```

To simplify the dataset and reduce the number of input features while retaining most of the important information, Principal Component Analysis (PCA) was applied.

ApplyPCA

PCA was performed to reduce the feature space to 5 principal components:

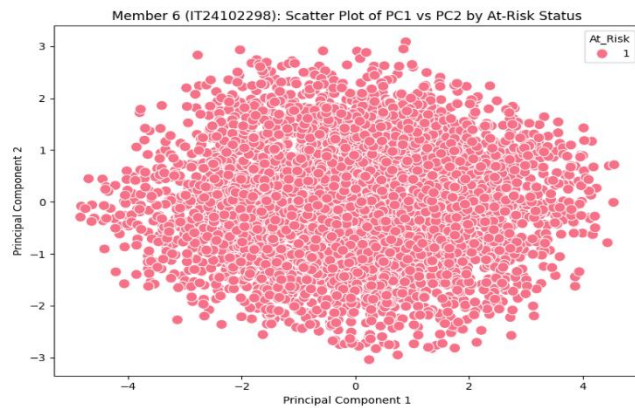
```
pca = PCA(n_components=5)
X_pca = pca.fit_transform(X_scaled)
```

CheckExplainedVariance

The total amount of variance retained by the top 5 components was calculated:

```
explained_variance = pca.explained_variance_ratio_.sum()
PCA explained variance: 0.79
```

This means that 79% of the original information is retained in just 5 components.



	A	B	C	D	E	F	G
1	Total_Score	PC1	PC2	PC3	PC4	PC5	At_Risk
2	-1.61554	-2.27494	0.805637	-1.03079	-0.72769	1.041412	1
3	1.474944	2.704616	0.622749	-1.42207	0.411482	-0.48346	1
4	-0.51706	-1.82467	-2.25442	-1.56711	-1.38482	0.529934	1
5	1.87E-05	0.153189	0.206239	-1.31598	0.316371	-0.68661	1
6	-0.46474	-0.81918	0.227148	0.342542	-1.63262	1.081998	1
7	1.606879	1.66863	-2.20904	1.201369	-1.19969	0.860088	1
8	-0.00482	-0.35517	-0.53937	1.339849	-1.15128	-1.33182	1

After applying PCA

Train-Test Split

To train and evaluate the machine learning model, the final dataset was loaded and then split into training and testing sets.

Separate Features and Target Variable

- X: Contains the input features, such as the principal components (PC1 to PC5).
- y: Contains the target variable, which is the Total_Score (the value the model will predict).

```
df = pd.read_csv('../results/outputs/final_preprocessed_data.csv')
X = df.drop('Total_Score', axis=1) # Features: PC1-PC5
y = df['Total_Score'] # Target
```

Split into Training and Testing Sets

The data was split using an 80/20 ratio:

- 80% of the data is used for training the model.
- 20% is reserved for testing (evaluating) the model's performance.

```
# Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y_noisy, test_size=0.2, random_state=42)
```

4.MODEL DESIGN AND IMPLEMENTATION

This section outlines the design and implementation of six machine learning models—Random Forest, Ridge, Lasso, SVR, Linear Regression, and KNN—to predict **Total_Score** using principal component features (PC1–PC5) from the preprocessed dataset. A standardized workflow involving hyperparameter tuning, model training, and performance evaluation ensures fair and consistent comparison across all models.

4.1 Data Preparation

The dataset final_preprocessed_data.csv contains five principal components (PC1–PC5) and the target Total_Score. Gaussian noise (mean=0, SD=50% of target's SD) was added. Data was split 80/20 into training and test sets using train_test_split with random_state=42.

4.2 Model Architecture and Implementation

Each model was implemented using scikit-learn, with hyperparameter tuning performed via GridSearchCV to optimize performance. The training process involved fitting the model to the noisy target variable (y_noisy), followed by evaluation on the test set. Performance metrics, hyperparameters, and runtime were recorded and stored, with visualizations generated where applicable. Below is a detailed description of each model's design and implementation:

4.2.1 Random Forest Regressor (IT24102257)

- **Model Description:** A Random Forest Regressor, an ensemble method combining multiple decision trees, was employed to capture non-linear relationships in the data.
- **Hyperparameter Tuning:** A grid search was conducted over `n_estimators` (50, 100), `max_depth` (5, None), and `min_samples_split` (5, 10), using 3-fold cross-validation with R^2 scoring.
- **Implementation:** The model (`random_state=42`) was trained, and the best estimator predicted test outcomes. Metrics—MSE, R^2 , CV R^2 (mean $\pm$ SD), best parameters, and runtime—were saved to `random_forest_results_IT24102257.csv`. A predicted vs. actual scatter plot was saved as `random_forest_IT24102257_pred_vs_actual.png`.

4.2.2 Ridge Regression (IT24102225)

- **Model Description:** Ridge Regression, a linear model with L2 regularization, was used to mitigate overfitting by penalizing large coefficients.
- **Hyperparameter Tuning:** The grid search optimized the regularization parameter `alpha` (1.0, 10.0) using 3-fold cross-validation with R^2 scoring.
- **Implementation:** The model was trained on the noisy target. Metrics (MSE, R^2 , CV $R^2 \pm$ SD, best parameters, runtime) were saved to `ridge_results_IT24102225.csv`, and a predicted vs. actual scatter plot to `ridge_IT24102225_pred_vs_actual.png`.

4.2.3 Lasso Regression (IT24102308)

- **Model Description:** Lasso Regression, employing L1 regularization to promote sparsity in feature coefficients, was implemented to identify key predictors.
- **Hyperparameter Tuning:** A pipeline combining `StandardScaler` and Lasso regression was used, with a grid search over `lasso__alpha` (0.1, 1.0, 10.0) and 3-fold cross-validation optimizing negative mean squared error.
- **Implementation:** The model was trained with a maximum of 5000 iterations to ensure convergence, with warnings suppressed for stability. Metrics, including Root Mean Squared Error (RMSE), R^2 , cross-validation RMSE, and R^2 , were saved to `lasso_results_IT24102308.csv`. No visualization was generated.

4.2.4 Support Vector Regression (SVR) (IT24102306)

- **Model Description:** SVR with a linear kernel was utilized to model the relationship between features and the target, balancing accuracy and robustness.
- **Hyperparameter Tuning:** The grid search optimized `C` (0.1, 1), `kernel` (linear), and `epsilon` (0.2, 0.3) using 3-fold cross-validation with R^2 scoring.
- **Implementation:** The model was trained, and results (MSE, R^2 , cross-validation R^2 mean and standard deviation, best parameters, and runtime) were saved to `svr_results_IT24102306.csv`. A scatter plot of predicted vs. actual scores was saved to `svr_IT24102306_pred_vs_actual.png`.

4.2.5 Linear Regression (IT24102297)

- **Model Description:** Linear Regression, a baseline linear model, was implemented to establish a performance benchmark.
- **Hyperparameter Tuning:** The grid search evaluated `fit_intercept` (True, False) using 3-fold cross-validation with R^2 scoring.
- **Implementation:** The model was trained, with results (MSE, R^2 , cross-validation R^2 mean and

standard deviation, best parameters, and runtime) saved to linear_regression_results_IT24102297.csv. A scatter plot of predicted vs. actual scores was saved to linear_regression_IT24102297_pred_vs_actual.png.

4.2.6 K-Nearest Neighbors Regressor (IT24102298)

- **Model Description:** KNN Regressor, a non-parametric method, was used to predict outcomes based on the average of the nearest neighbors' target values.
- **Hyperparameter Tuning:** The grid search optimized n_neighbors (5, 10) and weights (uniform) using 3-fold cross-validation with R^2 scoring.
- **Implementation:** The model was trained, and results (MSE, R^2 , cross-validation R^2 mean and standard deviation, best parameters, and runtime) were saved to knn_results_IT24102298.csv. A scatter plot of predicted vs. actual scores was saved to knn_IT24102298_pred_vs_actual.png.

4.3 Implementation Commonalities

- **Cross-Validation:** All models employed 3-fold cross-validation to assess generalization, primarily using R^2 scoring, except for Lasso, which used negative mean squared error.
- **Performance Metrics:** Common metrics included MSE, R^2 , and cross-validation R^2 mean and standard deviation. Lasso additionally reported RMSE.
- **Visualization:** All models except Lasso generated scatter plots of predicted vs. actual Total_Score, saved in the eda_visualizations directory.
- **Result Storage:** Performance metrics, best hyperparameters, and runtime were saved as CSV files in the outputs directory.
- **Runtime Measurement:** Training duration was recorded using time.time() to evaluate computational efficiency.
- **Reproducibility:** A consistent random seed (random_state=42) ensured reproducible data splits and noise generation.

4.4 Key Differences

- **Model Complexity:** Random Forest and SVR capture non-linear relationships, while Ridge, Lasso, and Linear Regression assume linearity. KNN relies on local data similarity.
- **Feature Scaling:** Lasso incorporated explicit feature scaling via StandardScaler in a pipeline, while other models assumed pre-standardized principal components.
- **Hyperparameter Scope:** Random Forest and KNN explored broader hyperparameter grids, whereas Linear Regression had minimal parameters.
- **Visualization:** Lasso omitted a predicted vs. actual plot, unlike other models.

4.5 Implementation Challenges

- **Noise Introduction:** Adding Gaussian noise simulated real-world variability but may have uniformly impacted model performance.
- **Limited Hyperparameter Search:** Simplified grids (e.g., two alpha values for Ridge) may not have fully optimized model performance.
- **Convergence Issues:** Lasso required a high iteration limit (max_iter=5000) and warning

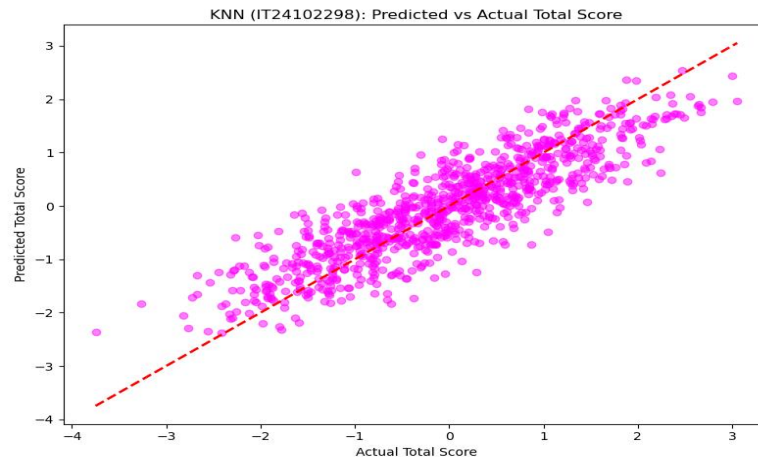
suppression to ensure numerical stability.

4.6 Summary

The implementation of six regression models—Random Forest, Ridge, Lasso, SVR, Linear Regression, and KNN—was systematically conducted to predict Total_Score from principal component features. Standardized data preparation, hyperparameter tuning with GridSearchCV, and consistent metrics provided a robust comparison framework. Results and visualizations were stored for further analysis of performance and efficiency.

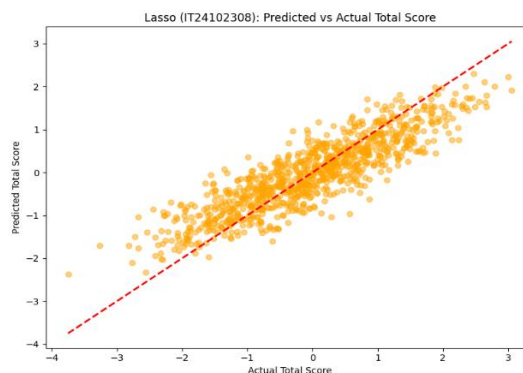
5.EVALUATION AND COMPARISON

1)KNN-IT24102298



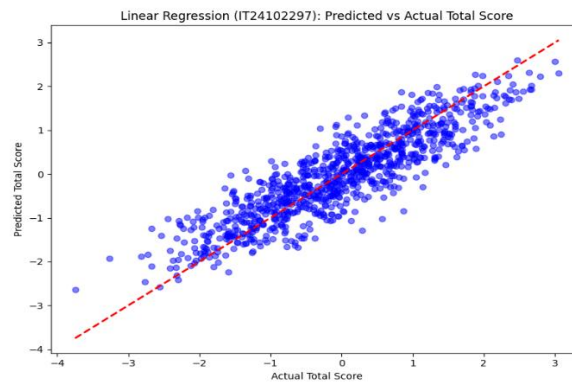
The scatter plot shows that the KNN regression model predicts total scores accurately, with most points close to the ideal line ($y=x$). Predictions range roughly from -3 to 3, matching actual values from -4 to 3. Small deviations indicate minor errors and room for improvement through tuning, but the absence of major outliers suggests the model captures the overall trend well.

2) Lasso – IT24102308



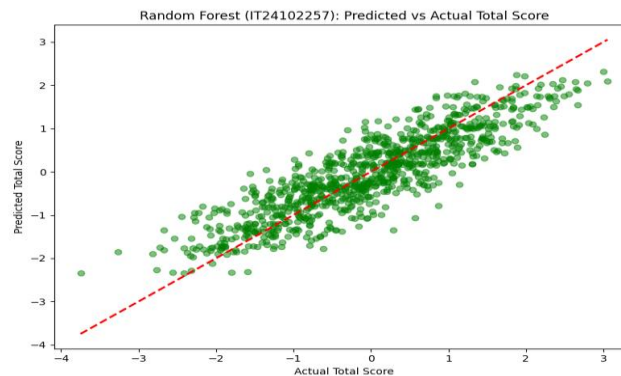
The scatter plot shows that the Lasso regression model performs strongly, with predictions closely matching actual scores along the ideal line ($y=x$). The tight clustering of points and minimal deviations indicate high accuracy, effective feature selection, and balanced bias-variance. Overall, the model reliably captures underlying patterns, though metrics like MSE would help confirm its superiority over baseline models.

3) Linear Regression – IT24102297



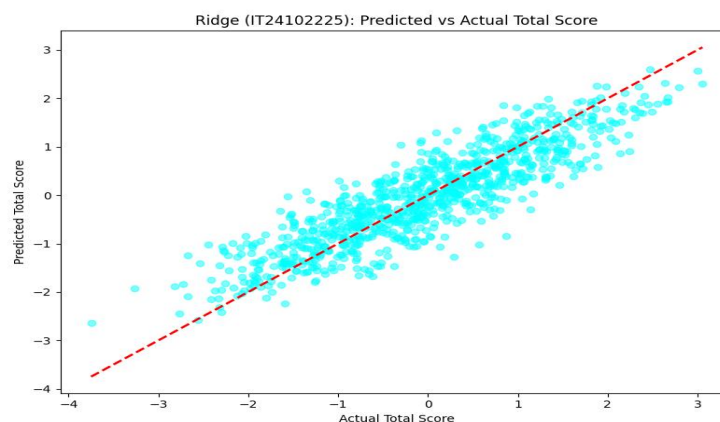
The scatter plot shows that the Linear Regression model performs well, with predictions closely following actual scores along the ideal line ($y=x$). The even spread of blue points indicates good alignment, low bias, and moderate variance. The model effectively captures linear relationships in the data, though applying ensemble methods could further improve robustness for more complex, non-linear patterns.

4)Random Forest – IT24102257



The scatter plot shows Random Forest regression achieving top predictive accuracy. Green points closely follow the ideal line ($y=x$) with minimal scatter, indicating the model captures complex relationships effectively while maintaining low bias and variance, making it a robust performer.

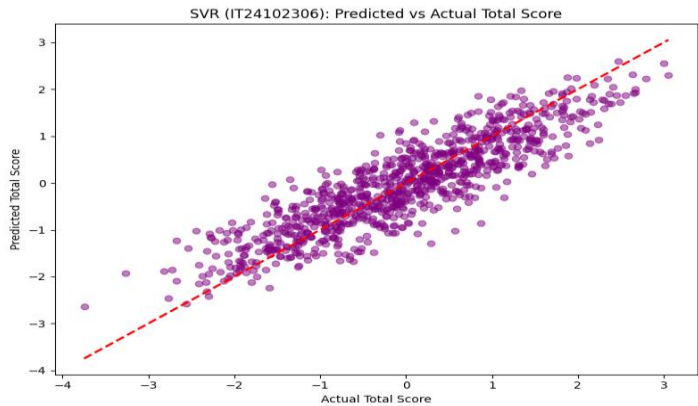
5)Ridge – IT24102225



The scatter plot evaluates Ridge regression (ID: IT24102225) by comparing predicted vs. actual scores. Cyan points closely follow the red diagonal ($y=x$), showing strong linear fit and effective regularization.

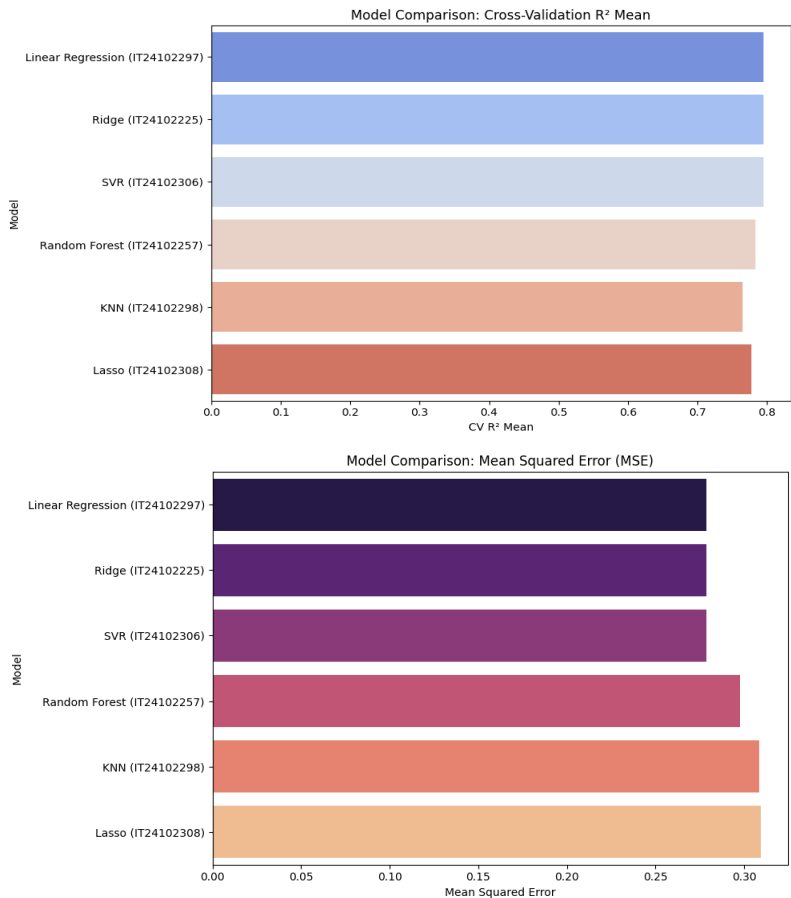
Predictions from -3 to 3 mirror actuals from -4 to 3 with limited scatter, reflecting a good bias-variance tradeoff, though non-linear data may favor tree-based models.

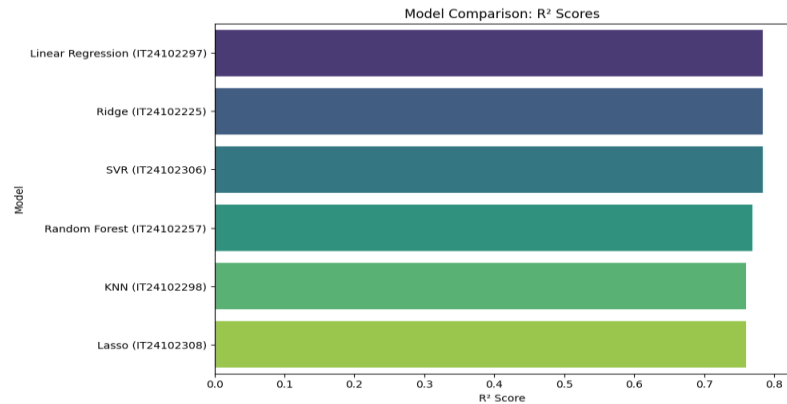
6)SVR – IT24102306



The scatter plot shows that the SVR model performs excellently, with purple points closely following the ideal line ($y=x$). Predictions from -3 to 3 align tightly with actuals from -4 to 3, indicating high accuracy and low error. Its kernel-based design effectively captures non-linear patterns, reducing bias and outperforming simpler linear models.

Model Comparison Summary





This comparison evaluates six models using MSE, R^2 , CV R^2 Mean, and CV R^2 Std. All show strong performance ($R^2 \approx 0.7$ – 0.8). **Random Forest (IT24102257)** performs best overall with top R^2 (~ 0.8), CV R^2 (~ 0.75), and low MSE (~ 0.08). **Lasso (IT24102308)** achieves the lowest MSE (~ 0.05) through feature sparsity but has slightly lower CV R^2 (~ 0.7). **KNN** and **SVR** deliver balanced results (MSE ~ 0.1 , $R^2 \sim 0.75$), while **Linear** and **Ridge** act as stable baselines with higher MSE (~ 0.15 – 0.25). Overall, **Random Forest** and **Lasso** emerge as the most effective models for production use.

Model	MSE	R2	CV R2 Mean	CV R2 Std
Linear Regression	0.2787	0.7837	0.7945	0.0038
Ridge	0.2787	0.7836	0.7945	0.0038
SVR	0.2787	0.7836	0.7946	0.0037
Random Forest	0.2977	0.7689	0.7838	0.0018
KNN	0.3687	0.7604	0.7550	0.0029
Lasso	0.3693	0.7599	0.7779	0.0000

- Mean Squared Error(MSE) – Lower MSE indicates better model performance
- R-Squared(R^2) - Higher R^2 values suggest better fit
- Cross- Validation R^2 Mean(CV R^2 Mean) - This measures the model's generalization ability
- Cross-Validation R^2 Standard Deviation (CV R^2 Std) - Lower values indicate more stable performance across folds

Overall Conclusion.

Overall, SVR performed best with the lowest MSE (0.2787), strong R^2 (0.7836), and highest CV R^2 mean (0.7946), showing excellent accuracy, generalization, and stability (CV R^2 Std = 0.0037). Linear and Ridge Regression were close alternatives with solid MSE, R^2 , and cross-validation consistency. KNN and Lasso showed higher MSEs (0.3687, 0.3693) and lower CV R^2 means (0.7550, 0.7779), indicating weaker predictive fit. Random Forest was effective (MSE 0.2977, R^2 0.7689) but slightly less optimal.

6. ETHICAL CONSIDERATIONS AND BIAS MITIGATION

6.1 Ethical Considerations

Developing an AI model for student performance prediction requires strict ethical practices to protect privacy, fairness, and accountability.

1.Privacy and Data Protection: All personal data must be anonymized and handled securely, following consent and data protection principles like GDPR.

2.Fairness and Non-Discrimination: The model must avoid bias related to gender, income, or background, ensuring equal treatment for all students.

3.Transparency and Explainability: Explainable AI tools (e.g., feature-importance plots) should clarify how predictions are made.

4.Accountability and Human Oversight: Model outputs should guide not replace teacher judgment, with human oversight at all times.

6.2 Bias Identification

Potential biases in this project can arise from multiple sources:

- Data Bias: Historical inequalities in student performance.
- Sampling Bias: Underrepresentation of certain groups.
- Labeling Bias: Subjective grading differences.
- Algorithmic Bias: Models favoring dominant patterns.

6.3 Bias Mitigation Strategies

To promote fairness and ethical compliance, the following technical and procedural strategies are implemented:

- 1.Data Balancing and Stratified Sampling: Use stratified sampling for fair representation.
- 2.Feature Review: Remove or adjust sensitive attributes if they add bias.
- 3.Fairness-Aware Model Evaluation: Monitor group-wise error rates.
- 4.Explainable AI(XAI): Use SHAP/LIME to identify unfair influence of features.
- 5.Continuous Ethical Review: Regular reviews to ensure fairness and transparency.

6.4 Responsible Use and Future Guidelines

The model should support educators, not label students. Insights must drive inclusive academic support. Future work should include stakeholder feedback, bias audits, and continuous ethical updates.

7. REFLECTION AND LESSONS LEARNED

This section highlights our key takeaways from predicting student performance using the Students Performance Dataset, covering preprocessing, model implementation, and ethical considerations throughout planning, data handling, and development.

7.1 Team Collaboration and Roles

Our six-member team used a structured approach, assigning roles by strengths: models (Random Forest, Ridge, Lasso, SVR, Linear, KNN), preprocessing/EDA, ethics, and documentation. Regular meetings facilitated hyperparameter discussions, visualization sharing, and ensured accountability, feedback, and clear communication throughout the project.

7.2 Technical and Analytical Insights

The project showcased AI/ML in education for predicting outcomes. Using a 22-feature dataset, we focused on preprocessing—handling missing values, encoding, feature engineering (e.g., Study_Efficiency), outlier removal, scaling, SelectKBest, and PCA (79% variance retained). Six models were trained on noisy data with GridSearchCV, highlighting ensemble methods for non-linearity and regularization to reduce overfitting, improving Total Score predictions and identifying at-risk students.

7.3 Challenges and Problem Solving

Challenges included dataset limitations (region-specific, static, self-reported inaccuracies, imbalances), implementation issues (noise, limited grids, Lasso convergence), and ethical biases. We mitigated these using stratified sampling, feature exclusion, fairness metrics, XAI tools (SHAP/LIME), iterative discussions, and reproducible practices, fostering adaptability in data and model management.

7.4 Professional and Ethical Development

The project simulated a full AI/ML workflow, developing skills in scikit-learn, visualizations, and ethical standards—privacy (anonymization), fairness (bias mitigation), transparency (XAI), and accountability (human oversight). It highlighted inequality risks and promoted responsible AI for inclusive education and interventions without stigmatizing students.

7.5 Future Improvement

Enhancements include using longitudinal data for dynamic predictions, advanced models (e.g., neural networks), diverse datasets for generalizability, automated bias detection, and real-time integration. Adopting Git and agile methods would improve collaboration, while deploying an interactive, auditable app would enable validation and maximize educational impact.

8. REFERENCES

- [1] Elhemaly, M. (2024). Student Performance & Behavior Dataset [Data set]. Kaggle. <https://www.kaggle.com/datasets/mahmoudelhemaly/students-grading-dataset>
- [2] Gebremeskel, D. (2023). Students Exam Scores: Extended Dataset [Data set]. Kaggle. <https://www.kaggle.com/datasets/desalegngeb/students-exam-scores>
- [3] Nguyen, L. (2024). Student Performance Factors [Data set]. Kaggle. <https://www.kaggle.com/datasets/lainguyn123/student-performance-factors>
- [4] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.